phone_app/lib/phone_app/conversations/conversations.ex

```
def send_sms_message(params = %Schema.NewMessage{}) do
  # This version of send_sms_message uses mock data, it doesn't
  #   make an HTTP request.
  #
  # Later, we will write a new version that sends an HTTP request
  #   to a mock SMS server.
  params = %{
    message_sid: "mock-" <> Ecto.UUID.generate(),
    account_sid: "mock",
    body: params.body,
    from: "mock",
    to: params.to,
    status: "mock",
    direction: :outgoing
  }

  create_sms_message(params)
end
```

Nothing is exciting here. We're simply calling the create_sms_message/1 function with mock data. We'll fill this out in the next chapter!

Try to fill out the form with various messages. You'll see that they get added to the conversation you're in. You can also press the large plus button at the top left[18] to create a new contact. Attempt to send invalid messages as well. You can use invalid phone numbers, or submit a message consisting of only spaces. Everything should work, and errors are presented back to you when something goes wrong.

Let's switch gears and end this section by looking at how our web server is started.

How Does Our Server Start?

In this chapter, you started a web server with iex -S mix phx.server. It's easy to take this for granted, but it's useful to recognize why a server starts and what port it starts on.

Phoenix uses configuration files to set up certain parts of the framework. If you look at the config/dev.exs file, you'll see a line that tells Phoenix to configure a server on port 4004:

phone_app/config/dev.exs

```
config :phone_app, PhoneAppWeb.Endpoint,
  http: [ip: {127, 0, 0, 1}, port: 4004],
```

18. http://localhost:4004/messages/new